

Homework 1: Branch Prediction

Information

- Для выполнения домашнего задания потребуется использовать симулятор ChampSim (<https://github.com/ChampSim/ChampSim/tree/master>) (master branch)
- В качестве default-ой конфигурации использовать Champsim JSON config, предоставленный в домашнем задании.
- Для оценки производительности в качестве бенчмарков нужно будет использовать часть трасс SPEC CPU 2017 из DPC-3, а именно
 - 600.perlbench_s-1273B.champsimtrace.xz
 - 602.gcc_s-1850B.champsimtrace.xz
 - 603.bwaves_s-2931B.champsimtrace.xz
 - 605.mcf_s-1536B.champsimtrace.xz
 - 607.cactuBSSN_s-2421B.champsimtrace.xz
 - 619.lbm_s-2676B.champsimtrace.xz
 - 620.omnetpp_s-141B.champsimtrace.xz
 - 621.wrf_s-575B.champsimtrace.xz
 - 623.xalancbmk_s-165B.champsimtrace.xz
 - 625.x264_s-12B.champsimtrace.xz
 - 627.cam4_s-490B.champsimtrace.xz
 - 628.pop2_s-17B.champsimtrace.xz
 - 631.deepsjeng_s-928B.champsimtrace.xz
 - 638.imagick_s-10316B.champsimtrace.xz
 - 641.leela_s-149B.champsimtrace.xz
 - 644.nab_s-12459B.champsimtrace.xz
 - 648.exchange2_s-387B.champsimtrace.xz
 - 649.fotonik3d_s-1176B.champsimtrace.xz
 - 654.roms_s-293B.champsimtrace.xz
 - 657.xz_s-4994B.champsimtrace.xz

Ссылка: <https://dpc3.compas.cs.stonybrook.edu/champsim-traces/спецспу/>
Занимаемый объем на диске примерно 10ГБ.

- В качестве результата замера производительности нужно будет использовать GMEAN значения IPC и branch mispredictions per Kilo instructions (mpki) на всем наборе трасс (предоставленном выше).

1 Markov Predictor vs Saturating Counters

- Реализуйте Markov Predictor (frequency counters) вместо битовых счетчиков в бимодальном предсказателе.
- Реализуйте вероятностный выбор предсказания в Markov Predictor (т.е. вместо значения с максимальной частотой выбирается предсказание на основе вероятностей $(\text{частота1} / (\text{частота1} + \text{частота2}))$)
- Сравните модели из пунктов 1 и 2 с исходным бимодальным предиктором.

2 Oracle

- Реализуйте идеальный **conditional** branch predictor с 0 mpki. Какой прирост IPC получился? Как далеко hashed perceptron от идеального предсказателя?

3 Two-level adaptive BPs

- Реализуйте схемы GAg и GAp. Сравните точность предсказания с бимодальным предиктором при одинаковых (близких) размерах предсказателей.

4 Path-based two-level BP

- Реализуйте схему GAp, однако вместо глобальной истории direction бранчей используйте path history (значения ip предыдущих n бранчей). Обратите внимание на функцию вычисления индекса обращения к РНТ таблице. Какая точность, производительность получились? Сравните с бимодальным предсказателем.

5 Analysis of hard-to-predict branches

- Встречаются ли в трассах бранчи, которые неверно предсказываются hashed perceptron в более чем 90% (и 70%) случаев? Какая часть

от общего числа ошибок предсказания приходится на такие случаи в среднем на наборе бенчмарков?

- (дополнительно) Как бы информация о том, какие ветки сложно-предсказываемые, могла бы помочь при компиляции программы?

6 Likely branches

- Встречаются ли в трассах ветки, которые не меняют направление перехода по ходу исполнения программы? Как с помощью SW/HW co-design оптимизации можно улучшить предсказание таких ветвей? Оцените эффективность такой оптимизации при использовании hashed perceptron.

7 Local histories

- Реализовать схемы PAr и PAg. Сравните точность предсказания с бимодальным предиктором при одинаковых (близких) размерах предсказателей

8 Per-set modules

- Реализовать схемы SAs и SAp. Придумайте, как организовать ветки в set-ы для ваших моделей. Сравните точность предсказания с бимодальным предиктором при одинаковых (близких) размерах предсказателей